Rather than depending on the user's discretion, which can lead to incorrect data due to accidental overwrites, Elasticsearch provides another endpoint: `_create`. This endpoint solves the overwrite situation. We can use the `_create` endpoint instead of `_doc` when indexing a document to avoid overriding the existing document. Let's look at this in action.

THE _CREATE API

Let's index the `movie` document with ID `100`, but this time let's use the `_create` endpoint. The next listing shows the operation's invocation.

Listing 5.5 Indexing a new movie using the `_create` endpoint

```
PUT movies/_create/100
{
  "title":"Mission: Impossible",
  "director":"Brian De Palma"
}
```

We've indexed a new movie (*Mission: Impossible*), this time using the `_create` endpoint instead of `_doc`. The fundamental difference between these two methods is that the `_create` method does not let us reindex the document with the same ID, while `_doc` wouldn't mind.

Next, let's try to change the content of the document by sending a tweet message as the `movie` document. The query is shown in next.

Listing 5.6 Adding and updating a field

```
PUT movies/_create/100
{
  "tweet":"A movie with popcorn is so cool!"
}
```

Indexing a tweet in place of an existing movie

We are (maybe accidentally) overriding the contents of the `movie` document. However, Elasticsearch throws a version conflict error:

```
{
"type" : "version_conflict_engine_exception",
"reason":"[100]:version conflict,document already exists(current version[1])"
}
```

Elasticsearch doesn't let the data be overwritten. This is the `_create` API's way of indicating that the document cannot be updated because the version already exists.

> **NOTE** Although the `_create` endpoint does not allow us to update the document, we can swap the `_create` endpoint with `_doc` to perform the update if needed.

The takeaway for this section is that if we need to protect our documents by not allowing them to be overwritten accidentally, we should use the `_create` API.